JAVA GAME PROJECT

Developer Portfolio / Project Case Study

Strategic Fighting & Character-Collection Game
A Java-based combat system combining card-driven battles, tactical switching, character-specific mechanics, summoning, progression, and persistent game data.

Implementation	Java
Source file	Main(5).java
Source size	8,878 lines
Project focus	Combat • AI • Collection • Progression • Balancing

Prepared as a professional portfolio document. The content is based on the supplied Java source code.

1. Project Overview

This project is a Java-based strategic fighting and character-collection game. The implementation combines a real-time-style combat flow with a card system, team management, character progression, themed banners, weighted summoning, and persistent inventory data. The code is designed around a reusable Fighter model while allowing individual characters to override combat behaviour for their own mechanics.

The project goes beyond a simple battle simulator by connecting combat, progression, probability, AI decision-making, and persistent player state into one system. Teams contain three fighters, while combat uses Strike, Blast, Quirk, Special Move, and Ultimate cards. The combat loop also manages Ki, Vanish, card generation, switching, cover changes, status effects, and defeat handling.

Project Highlights

- Object-oriented Java combat architecture with a reusable Fighter base model.
- Five-card combat vocabulary: Strike, Blast, Quirk, Special Move, and Ultimate.
- Three-fighter team system with switching, cover changes, defeat handling, and momentum resets.
- Character-specific behaviour implemented through overridden combat methods.
- AI-style tactical switching based on health, available cards, opponent Ki, and pressure.
- Weighted summon banners with headliners, featured characters, and hard pity.
- Limit Break progression using duplicate character copies.
- Persistent coins, pity counters, and inventory using a properties-based save file.
- Tier and Featured Boost systems designed to improve competitive balance.

Source Metrics

Metric	Observed in supplied source
Source lines	8,878
Java enum declarations	4
Class declarations	36
@Override methods	209
Banner definitions	10
Character tier entries	28
Limit Break character choices	28

2. Technical Architecture

The source is structured around a central Main driver plus reusable domain types such as Card, NeutralAttack, AttackRange, CharacterColor, Banner, SummonSave, Teams, and Fighter-derived character classes. This allows common combat behaviour to live in shared logic while individual fighters extend or override that behaviour.

Core Architecture

Layer	Responsibility
Game driver	Timer, battle flow, combat progression, match results
Fighter model	HP, Ki, Vanish, cards, combat actions, statuses, character state
Team model	Three-fighter roster, active/bench state, switching and hand management
Combat engine	Combo execution, cover changes, Danger Sense, timing and defeat handling
Summon system	Banners, weights, pity, copies, team selection
Progression	Limit Break, tier boosts, Featured Boosts
Persistence	Coins, pity counters and character inventory

Combat System

The combat engine advances a shared timer and uses action time to recover Ki and Vanish, reduce switch cooldowns, and progress card drawing. It supports combo execution, Ultimate prioritisation, card availability checks, cover changes, Danger Sense, and defeat processing. The system also applies character colour advantages and supports temporary effects and defensive interactions.

> The colour system includes advantage, disadvantage, and immunity handling. An advantageous attack normally uses a 1.50 damage multiplier, with a stronger-colour variant capable of reaching 1.75, while disadvantage reduces damage to 0.75.

AI / Tactical Decision-Making

The project contains decision logic for tactical switching rather than switching randomly on every action. A candidate fighter is scored using health percentage, whether the fighter has a playable card, the active fighter's relative health, and the opponent's Ki. Switching can be triggered by low health, disabled status, or pressure when the active fighter lacks a playable card.

3. Collection, Summoning & Progression

The collection layer turns combat results into a persistent progression loop. Players earn coins from matches, spend them on summons, collect duplicate copies, build an inventory, and select three different fighters for a team.

Banner System

- A standard banner can use the complete summon pool with character-specific rates.
- Themed banners define their own weighted pool.
- Characters can be marked as normal, featured, or headliner.
- Relative weights are converted into effective summon probabilities.
- Headliners can trigger an additional Featured Boost in combat.

Pity & Weighted Summoning

The implementation uses weighted random selection and a banner-specific pity counter. The configured hard pity is 30 summons without obtaining the banner headliner. When the pity threshold is reached, the headliner is selected directly instead of using the normal weighted roll.

Limit Break Progression

Duplicate copies contribute to a seven-copy Limit Break progression. Different copy thresholds compound damage, defense, and HP modifiers, with character-specific choices controlling whether certain upgrades favour damage or defense. Copies above the configured cap are converted into coins instead of being wasted.

> The source applies Limit Break first and then applies the character's strength tier and Featured Boost. This creates a layered progression model rather than a single flat bonus.

Balance System

Characters are assigned strength tiers from W through C. The supplied configuration gives larger damage and defense multipliers to lower-ranked characters, while stronger characters receive smaller or zero compensation. A separate Featured Boost is applied to banner headliners, keeping banner status independent from the character's underlying strength tier.

Tier	Damage bonus	Defense bonus
W	+500%	+450%
X	+400%	+350%
Z	+300%	+250%
S	+240%	+200%
A	+150%	+120%
B	+80%	+50%
C	+0%	+0%
Featured	+30%*	+30%*

* Featured Boost is applied to banner headliners on top of their strength-tier multiplier.

4. Character-Specific Mechanics

A major strength of the project is that fighters are not limited to identical statistics. The source uses overridden methods to give individual characters their own rules and state. This supports mechanics such as transformations, gauges, resource manipulation, temporary buffs, debuffs, healing, bleed/faint effects, and special interactions with the combat engine.

Examples from the supplied implementation

- **Janemba:** builds a dimensional-pressure cycle through attacks; his main ability restores resources, unlocks an Ultimate, increases damage/defense, and disrupts the opponent's defense and Ki.
- **Vegito / fusion-style fighters:** the architecture supports character-specific transformations and power-state changes through overridden abilities.
- **Cover-focused fighters:** the combat engine allows a benched fighter to take a matching card type and then follow the cover change with a configured combo card.
- **Gauge / hit-based fighters:** the Fighter design permits custom counters and state variables that can trigger effects after particular combat events.
- **Resource-disruption fighters:** character logic can alter enemy Ki, cards, defensive state, or temporary buffs.

Persistence & Player State

The project uses Java's Properties-based file persistence through a dedicated SummonSave component. Coins, pity counters, and character copies are loaded from and saved to a local save file, allowing collection progress to continue between matches. The system also includes a reset operation for clearing stored progress.

Match Economy

The supplied configuration starts a new team with 6,000 coins. A single summon costs 100 coins, while a ten-summon bundle costs 900. Match rewards are 1,000 coins for a win, 500 for a loss, and 200 for a draw. These values are implemented as tunable constants rather than hard-coded throughout the battle logic.

5. Engineering Skills Demonstrated

- **Object-Oriented Programming:** reusable base classes, inheritance, method overriding, cloning, and polymorphic fighter behaviour.
- **State Management:** combat state, timers, statuses, resources, switch cooldowns, gauges, and progression state.
- **Algorithms & Logic:** weighted random selection, pity guarantees, candidate scoring, conditional switching, and layered multipliers.
- **Game-System Design:** interconnected combat, economy, collection, progression, and balancing systems.
- **Persistence:** local storage of coins, pity counters, and inventory data.
- **Defensive Programming:** null checks, input validation, invalid-banner detection, clone validation, and boundary handling.
- **Extensibility:** banners and character balance values are represented through configurable structures, making future content easier to add.

6. Technical Challenges & Solutions

Challenge	Implemented approach
Different characters need different behaviour	Shared Fighter architecture with overridden combat methods and character-specific state.
Balance characters of different strength	Separate strength tiers plus independent Featured Boosts and Limit Break multipliers.
Keep summons fair	Weighted rolls combined with banner-specific pity guarantees.
Prevent progress loss	Persistent Properties-based save system for coins, pity and inventory.
Make AI switching meaningful	Candidate scoring based on health, playable cards and opponent pressure.
Handle complex combo interactions	Dedicated combo engine with cover changes, follow-ups, Danger Sense and defeat handling.

7. Future Development

Potential next steps include separating the large source file into focused packages/classes, adding automated unit tests for damage and balance calculations, expanding AI behaviour, improving UI presentation, adding analytics for battle outcomes, and introducing more systematic balance simulations. These changes would improve maintainability and make future content easier to develop and test.

8. Conclusion

This project demonstrates the development of a sizeable Java game system rather than a single isolated feature. It combines object-oriented character design, combat algorithms, AI-style decision logic, probability systems, progression, persistence, and balancing into one connected application. The architecture also provides room for additional fighters, banners, abilities, and gameplay systems to be added over time.

Portfolio note: This document intentionally focuses on engineering decisions and implemented systems so the project can be presented to recruiters as a software-development project.